EfficientNet-B0: A Comprehensive Technical Manual

Deep Learning Architecture Documentation

February 2, 2026

Contents

1 Introduction

EfficientNet is a family of convolutional neural networks that achieve state-of-the-art accuracy while being significantly more efficient than previous models. The key innovation is **compound scaling**, which uniformly scales network width, depth, and resolution with a carefully balanced set of coefficients.

1.1 Key Contributions

- Compound scaling method that uniformly scales all dimensions
- Mobile Inverted Bottleneck Convolution (MBConv) blocks with Squeeze-and-Excitation
- Superior parameter efficiency compared to previous architectures
- EfficientNet-B0: 5.3M parameters, 77.1% top-1 accuracy on ImageNet

2 CNN Foundations (So It Is Not a Black Box)

This section summarizes the minimal set of CNN concepts needed to "read" EfficientNet-B0 like an engineering design rather than a mysterious function.

2.1 Tensors and the Convolutional Feature Hierarchy

A standard image tensor is $X \in \mathbb{R}^{H \times W \times C}$ where H, W are spatial dimensions and C is the number of channels (e.g., $C = 3$ for RGB). A CNN transforms X into progressively more abstract representations:

- early layers: edges, corners, color blobs
- mid layers: textures and repeated motifs
- late layers: object parts and class-specific compositions

2.2 Convolution: What a "Filter" Actually Does

A 2D convolution layer with C_{in} input channels and C_{out} output channels learns C_{out} filters. Each filter is a 3D tensor $W^{(o)} \in \mathbb{R}^{k \times k \times C_{\text{in}}}$. For an output channel o at spatial location (i, j):

$$Y_o(i,j) = \sum_{u=1}^{k} \sum_{v=1}^{k} \sum_{c=1}^{C_{\text{in}}} W^{(o)}(u,v,c)\, X(i+u, j+v, c) + b_o \tag{1}$$

2.3 Stride, Padding, and Output Shapes

For kernel size k, stride s, and padding p (per side), the output resolution is:

$$H_{\text{out}} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1, \quad W_{\text{out}} = \left\lfloor \frac{W + 2p - k}{s} \right\rfloor + 1 \tag{2}$$

Interpretation:

- Stride $s > 1$ downsamples (reduces spatial size) and increases the effective receptive field faster.
- Padding controls border effects; "same" padding (common in modern CNNs) keeps spatial size roughly unchanged when $s = 1$.

2.4 Pooling vs. Strided Convolution

Downsampling can be performed by pooling (e.g., max pooling) or by strided convolutions. EfficientNet largely uses strided convolutions / strided depthwise convolutions inside MBConv stages.

2.5 Receptive Field vs. Effective Receptive Field

The theoretical receptive field computed from kernel sizes and strides can be large, but the *effective* receptive field (where gradients are strongest) is often smaller and centered. EfficientNet-B0 still achieves global context via multiple downsampling stages and a final global average pool.

2.6 Why Batch Normalization Is Everywhere

Batch normalization (BN) stabilizes optimization by normalizing intermediate activations and learning affine re-scaling. A practical transfer-learning note: during fine-tuning, BN statistics can drift if your batch size is small or your new dataset distribution differs strongly.

2.7 Residual Connections and "Skip" Logic

Residual connections help gradients flow through deep networks. In MBConv, the skip connection is used only when the spatial size and channel count match (stride = 1 and $C_{\text{in}} = C_{\text{out}}$).

3 Compound Scaling

3.1 Motivation

Traditional approaches scale CNNs arbitrarily:

- **Depth scaling**: Adding more layers (e.g., ResNet-50 $\rightarrow$ ResNet-101)
- **Width scaling**: Adding more channels (e.g., WideResNet)
- **Resolution scaling**: Using higher resolution inputs

EfficientNet proposes a principled approach to scale all three dimensions.

3.2 Mathematical Formulation

The compound scaling formula is defined as:

$$\text{depth:} \quad d = \alpha^\phi \tag{3}$$

$$\text{width:} \quad w = \beta^\phi \tag{4}$$

$$\text{resolution:} \quad r = \gamma^\phi \tag{5}$$

Subject to the constraint:

$$\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2 \tag{6}$$

where:

- α, β, γ are constants determined by grid search
- ϕ is a user-specified coefficient that controls available resources
- The constraint ensures that for any ϕ, total FLOPS increase by $\approx 2^\phi$

3.3 EfficientNet-B0 to B7

For the baseline EfficientNet-B0:

$$\alpha = 1.0, \quad \beta = 1.0, \quad \gamma = 1.0, \quad \phi = 0 \tag{7}$$

For larger variants (B1-B7), the constants are:

$$\alpha = 1.2, \quad \beta = 1.1, \quad \gamma = 1.15 \tag{8}$$

4 Architecture Components

4.1 Swish Activation Function

The Swish activation function is defined as:

$$\text{Swish}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \tag{9}$$

where $\sigma(x)$ is the sigmoid function.
Properties:

- Smooth and non-monotonic
- Self-gated (output depends on input value)
- Better gradient flow than ReLU for deep networks
- Derivative: $\frac{d}{dx}\text{Swish}(x) = \text{Swish}(x) + \sigma(x)(1 - \text{Swish}(x))$

4.2 Squeeze-and-Excitation (SE) Block

SE blocks perform channel-wise feature recalibration through three operations:

1. Squeeze: Global average pooling

$$z_c = \frac{1}{H \times W} \sum_{i=1}^{H} \sum_{j=1}^{W} x_c(i, j) \tag{10}$$

2. Excitation: Two fully-connected layers

$$s = \sigma(W_2 \cdot \delta(W_1 \cdot z)) \tag{11}$$

where:

- $W_1 \in \mathbb{R}^{\frac{C}{r} \times C}$ reduces dimensionality by ratio r
- $W_2 \in \mathbb{R}^{C \times \frac{C}{r}}$ restores dimensionality
- δ is ReLU activation
- σ is sigmoid activation
- r is the reduction ratio (typically 0.25 in EfficientNet-B0)

3. Scale: Channel-wise multiplication

$$\tilde{x}_c = s_c \cdot x_c \tag{12}$$

4.3 Mobile Inverted Bottleneck Convolution (MBConv)

EfficientNet-B0 is mostly "a stack of MBConv blocks," so understanding MBConv removes most of the black-box feeling.

4.3.1 Why "Inverted" Bottleneck?

Classic bottlenecks (e.g., ResNet) reduce channels in the middle for efficiency. MobileNetV2-style inverted bottlenecks do the opposite:

- expand channels to a higher-dimensional space (cheap with 1×1 conv)
- apply an efficient spatial operator (depthwise conv)
- project back down to the desired output channels

4.3.2 Block Dataflow (High-Level)

Let input be $X \in \mathbb{R}^{H \times W \times C_{\text{in}}}$. The block computes:

$$X \xrightarrow{\text{Expand}} X_{\text{exp}} \xrightarrow{\text{Depthwise}} X_{\text{dw}} \xrightarrow{\text{SE}} X_{\text{se}} \xrightarrow{\text{Project}} X_{\text{proj}} \xrightarrow{\text{Skip (if allowed)}} X_{\text{out}} \tag{13}$$

4.3.3 Where the Nonlinearities Live

A common (and important) detail: EfficientNet-style MBConv typically uses nonlinearity (Swish/SiLU) after expansion and after depthwise convolution, but the final projection is often linear. This preserves information when projecting back to a lower dimension.

The MBConv block consists of:

4.3.4 Expansion Phase

$$X_{\text{exp}} = \text{Conv}_{1\times1}(X) \cdot t \tag{14}$$

where t is the expansion ratio (typically 6).

4.3.5 Depthwise Convolution

$$X_{\text{dw}} = \text{DWConv}_{k\times k}(X_{\text{exp}}) \tag{15}$$

where $k \in \{3, 5\}$ depending on the stage.

4.3.6 Projection Phase

$$X_{\text{proj}} = \text{Conv}_{1\times1}(X_{\text{se}}) \tag{16}$$

4.3.7 Skip Connection

If stride = 1 and input channels = output channels:

$$X_{\text{out}} = X + \text{DropConnect}(X_{\text{proj}}) \tag{17}$$

4.4 Stochastic Depth (Drop Connect)

Drop Connect randomly drops entire residual branches during training:

$$X_{\text{out}} = \begin{cases} X + \frac{X_{\text{proj}}}{1-p} \cdot M & \text{training} \\ X + X_{\text{proj}} & \text{inference} \end{cases} \tag{18}$$

where:

- $M \sim \text{Bernoulli}(1-p)$ is a binary mask
- p linearly increases from 0 to $p_{\max}$ across blocks
- Helps with gradient flow and acts as regularization

5 EfficientNet-B0 Architecture

5.1 Overall Structure

5.2 Step-by-Step Forward Pass (Math + Code)

This section shows the computation performed by each major stage, with both the mathematical operation and a minimal implementation sketch. Let input $X \in \mathbb{R}^{224\times224\times3}$.

Stage	Operator	Resolution	Channels	Layers	Stride	Kernel
0	Conv3×3	224^2	32	1	2	3
1	MBConv1, k3×3	112^2	16	1	1	3
2	MBConv6, k3×3	112^2	24	2	2	3
3	MBConv6, k5×5	56^2	40	2	2	5
4	MBConv6, k3×3	28^2	80	3	2	3
5	MBConv6, k5×5	14^2	112	3	1	5
6	MBConv6, k5×5	14^2	192	4	2	5
7	MBConv6, k3×3	7^2	320	1	1	3
8	Conv1×1	7^2	1280	1	1	1
9	Pooling + FC	1^2	1000	1	-	-

Table 1: EfficientNet-B0 Architecture Details

5.2.1 (0) Stem: 3×3 Convolution with Stride 2

Math (single output channel):

$$Y_o(i,j) = \sum_{u=1}^{3}\sum_{v=1}^{3}\sum_{c=1}^{3} W^{(o)}(u,v,c)\, X(2i+u, 2j+v, c) + b_o \tag{19}$$

Output tensor: $Y \in \mathbb{R}^{112\times112\times32}$.

Code (PyTorch-like pseudocode):

```
# X: [B, 3, 224, 224]
Y = Conv2d(3, 32, kernel_size=3, stride=2, padding=1)(X)
Y = BatchNorm2d(32)(Y)
Y = Swish()(Y)
# Y: [B, 32, 112, 112]
```

5.2.2 (1) MBConv Block: Expand → Depthwise → SE → Project

Let the input be $X \in \mathbb{R}^{H\times W\times C}$.

Expand (pointwise 1×1):

$$X_{\text{exp}} = \text{Swish}(\text{BN}(W_{1\times1} * X)) \in \mathbb{R}^{H\times W\times(tC)} \tag{20}$$

Depthwise convolution ($k \times k$):

$$X_{\text{dw}}(:,:,c) = \text{Swish}(\text{BN}(W^{(c)}_{k\times k} * X_{\text{exp}}(:,:,c))) \tag{21}$$

Squeeze-and-Excitation:

$$z_c = \frac{1}{HW}\sum_{i=1}^{H}\sum_{j=1}^{W} X_{\text{dw}}(i,j,c) \tag{22}$$

$$s = \sigma\big(W_2\,\delta(W_1 z)\big) \tag{23}$$

$$X_{\text{se}}(i,j,c) = s_c\, X_{\text{dw}}(i,j,c) \tag{24}$$

Project (pointwise 1×1, usually linear):

$$X_{\text{proj}} = \text{BN}(W'_{1\times1} * X_{\text{se}}) \in \mathbb{R}^{H'\times W'\times C'} \tag{25}$$

Skip (only if stride $= 1$ and $C = C'$):

$$X_{\text{out}} = X + \text{DropConnect}(X_{\text{proj}}) \tag{26}$$

Code (block sketch):

```
# Expand
Xe = Conv2d(C, t*C, 1, stride=1, padding=0)(X)
Xe = BN(t*C)(Xe)
Xe = Swish()(Xe)

# Depthwise
Xd = DepthwiseConv2d(t*C, kernel_size=k, stride=s, padding=k//2)(Xe)
Xd = BN(t*C)(Xd)
Xd = Swish()(Xd)

# Squeeze-Excite
z = GlobalAvgPool2d()(Xd) # [B, t*C]
z = Linear(t*C, (t*C)//r)(z); z = ReLU()(z)
z = Linear((t*C)//r, t*C)(z); s = Sigmoid()(z)
Xse = Xd * s.view(B, t*C, 1, 1)

# Project
Xp = Conv2d(t*C, C_out, 1, stride=1, padding=0)(Xse)
Xp = BN(C_out)(Xp)

# Skip
Y = X + DropConnect(p)(Xp) if (s==1 and C==C_out) else Xp
```

5.2.3 (2) Stage-by-Stage Shape Trace (EfficientNet-B0)

```
# Shapes shown as [C, H, W]
X = [ 3, 224, 224]
S0 = [ 32, 112, 112] # stem conv3x3, stride2
S1 = [ 16, 112, 112] # MBConv1 k3 s1 x1
S2 = [ 24, 56, 56] # MBConv6 k3 s2 x2
S3 = [ 40, 28, 28] # MBConv6 k5 s2 x2
S4 = [ 80, 14, 14] # MBConv6 k3 s2 x3
S5 = [112, 14, 14] # MBConv6 k5 s1 x3
S6 = [192, 7, 7] # MBConv6 k5 s2 x4
S7 = [320, 7, 7] # MBConv6 k3 s1 x1
H = [1280, 7, 7] # head conv1x1
```

5.2.4 (3) Head, Global Average Pooling, and Classifier

Head conv 1×1:

$$H = \text{Swish}(\text{BN}(W_{1\times1} * X_{\text{last}})) \in \mathbb{R}^{7\times7\times1280} \tag{27}$$

Global average pooling:

$$z_c = \frac{1}{49} \sum_{i=1}^{7} \sum_{j=1}^{7} H(i, j, c) \in \mathbb{R}^{1280} \tag{28}$$

Linear classifier (ImageNet 1000-way):

$$\text{logits} = Wz + b \in \mathbb{R}^{1000}, \quad p(y = k \mid x) = \frac{e^{\text{logits}_k}}{\sum_{m=1}^{1000} e^{\text{logits}_m}} \tag{29}$$

Code (end of network):

```
H = Conv2d(320, 1280, 1)(X_last)
H = BN(1280)(H)
H = Swish()(H)

z = H.mean(dim=(2, 3)) # global average pool -> [B, 1280]
logits = Linear(1280, 1000)(z)
probs = Softmax(dim=1)(logits)
```

5.3 Parameter Count Analysis

Total parameters: **5.3 million**

- Stem (Conv3×3): $3 \times 32 \times 3 \times 3 = 864$
- MBConv blocks: $\approx 4.2M$
- Head (Conv1×1): $320 \times 1280 = 409,600$
- Classifier: $1280 \times 1000 = 1.28M$

5.4 Computational Complexity

For input size $224 \times 224 \times 3$:

- FLOPs: $\approx 0.39 \times 10^9$ (390M FLOPs)
- Multiply-Adds: $\approx 0.195 \times 10^9$

6 Mathematical Derivations

6.1 Depthwise Separable Convolution

Standard convolution computational cost:

$$\text{Cost}_{\text{std}} = H \times W \times C_{\text{in}} \times C_{\text{out}} \times k^2 \tag{30}$$

Depthwise separable convolution cost:

$$\text{Cost}_{\text{dw}} = H \times W \times C_{\text{in}} \times k^2 + H \times W \times C_{\text{in}} \times C_{\text{out}} \tag{31}$$

Reduction factor:

$$\frac{\text{Cost}_{\text{dw}}}{\text{Cost}_{\text{std}}} = \frac{1}{C_{\text{out}}} + \frac{1}{k^2} \tag{32}$$

For $C_{\text{out}} = 256$ and $k = 3$:

$$\frac{\text{Cost}_{\text{dw}}}{\text{Cost}_{\text{std}}} = \frac{1}{256} + \frac{1}{9} \approx 0.115 \quad (8.7\times \text{ reduction}) \tag{33}$$

6.2 Receptive Field Calculation

For a stack of n layers with kernel size k and stride s:

$$RF = 1 + \sum_{i=1}^{n}(k_i - 1)\prod_{j=1}^{i-1} s_j \tag{34}$$

For EfficientNet-B0's final receptive field:

$$RF \approx 427 \times 427 \text{ pixels} \tag{35}$$

This is larger than the input size (224×224), ensuring global context.

7 Training Details

7.1 Optimization

- **Optimizer:** RMSprop with decay 0.9, momentum 0.9
- **Learning Rate:** 0.256, decayed by 0.97 every 2.4 epochs
- **Batch Size:** 2048 (distributed across multiple devices)
- **Weight Decay:** 1×10^{-5}
- **Epochs:** 350

7.2 Regularization

- **Stochastic Depth:** $p_{\max} = 0.2$
- **Dropout:** 0.2 before final classifier
- **Label Smoothing:** $\epsilon = 0.1$
- **Data Augmentation:** AutoAugment

7.3 Loss Function

Cross-entropy with label smoothing:

$$\mathcal{L} = -\sum_{i=1}^{K} y'_i \log(\hat{y}_i) \tag{36}$$

where the smoothed labels are:

$$y'_i = \begin{cases} 1 - \epsilon & \text{if } i = \text{true class} \\ \frac{\epsilon}{K-1} & \text{otherwise} \end{cases} \tag{37}$$

8 Implementation Considerations

8.1 Batch Normalization

BatchNorm statistics for layer l with batch $\mathcal{B}$:

Mean:

$$\mu_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} x \tag{38}$$

Variance:

$$\sigma^2_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} (x - \mu_{\mathcal{B}})^2 \tag{39}$$

Normalization:

$$\hat{x} = \frac{x - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \tag{40}$$

Scale and Shift:

$$y = \gamma \hat{x} + \beta \tag{41}$$

8.2 Weight Initialization

Kaiming He initialization for Conv layers:

$$W \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{out}}}}\right) \tag{42}$$

where n_{out} is the number of output channels.

For Linear layers:

$$W \sim \mathcal{N}(0, 0.01) \tag{43}$$

9 Performance Metrics

9.1 ImageNet Results

Model	Params (M)	FLOPs (B)	Top-1 Acc	Top-5 Acc
ResNet-50	25.6	4.1	76.0%	93.0%
MobileNetV2	3.5	0.3	72.0%	91.0%
EfficientNet-B0	5.3	0.39	77.1%	93.3%
EfficientNet-B7	66.0	37.0	84.4%	97.1%

Table 2: Comparison with Other Architectures

9.2 Efficiency Analysis

Parameter efficiency compared to ResNet-50:

$$\text{Efficiency} = \frac{\text{Accuracy}_{\text{EfficientNet-B0}}}{\text{Params}_{\text{EfficientNet-B0}}} \div \frac{\text{Accuracy}_{\text{ResNet-50}}}{\text{Params}_{\text{ResNet-50}}} \tag{44}$$

$$\text{Efficiency} = \frac{77.1/5.3}{76.0/25.6} = \frac{14.55}{2.97} \approx 4.9\times \tag{45}$$

10 Transfer Learning

10.1 Feature Extraction

Extract features from the layer before classification:

$$f(x) = \text{GlobalAvgPool}(\text{Head}(\text{MBConv blocks}(\text{Stem}(x)))) \tag{46}$$

Output: $f(x) \in \mathbb{R}^{1280}$

10.2 Fine-tuning Strategy

1. **Phase 1:** Freeze backbone, train classifier only
   - Learning rate: 1×10^{-3}
   - Epochs: 10-20
2. **Phase 2:** Unfreeze all layers, fine-tune end-to-end
   - Learning rate: 1×10^{-4} to 1×10^{-5}
   - Use differential learning rates (lower for early layers)
   - Epochs: 20-50

10.3 Recommended Learning Rates by Layer

$$\text{LR}(\text{layer}_i) = \text{LR}_{\text{base}} \times \lambda^{(N-i)/N} \tag{47}$$

where:

- $\lambda \in [0.1, 0.5]$ is the discriminative factor
- N is the total number of layer groups
- i is the layer group index (0 = stem, N = classifier)

11 Practical Tips

11.1 Data Preprocessing

Training:

- Random crop to 224×224
- Random horizontal flip (p=0.5)
- AutoAugment policy
- Normalize: mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]

Inference:

- Resize to 256×256
- Center crop to 224×224
- Normalize with same statistics

11.2 Memory Optimization

For limited GPU memory:

- Use mixed precision training (FP16)
- Gradient checkpointing for deep blocks
- Reduce batch size and use gradient accumulation:

$$\text{Effective batch size} = \text{batch size} \times \text{accumulation steps} \tag{48}$$

11.3 Common Issues and Solutions

Issue	Solution
Overfitting	Increase dropout, use stronger data augmentation, reduce model size
Slow convergence	Increase learning rate, reduce regularization, check data preprocessing
NaN losses	Reduce learning rate, use gradient clipping, check for data issues
Poor transfer learning	Unfreeze more layers, use lower learning rate, train longer

Table 3: Troubleshooting Guide

12 Conclusion

EfficientNet-B0 demonstrates that careful architectural design and compound scaling can achieve superior performance with fewer parameters. Key takeaways:

1. **Compound Scaling:** Balancing width, depth, and resolution is crucial
2. **MBConv Blocks:** Efficient building blocks with SE attention
3. **Parameter Efficiency:** 4.9× more efficient than ResNet-50
4. **Transfer Learning:** Excellent backbone for downstream tasks

13 References

- Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. ICML 2019.
- Hu, J., Shen, L., & Sun, G. (2018). Squeeze-and-Excitation Networks. CVPR 2018.
- Sandler, M., et al. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. CVPR 2018.
- Huang, G., et al. (2016). Deep Networks with Stochastic Depth. ECCV 2016.